

Google BigQuery / Dremel – Lecture Notes

Ekaterina Bochvaroska - 89252018

These notes combine my handwritten notes from lecture on 26.11.2025 with *Google BigQuery/Dremel* lecture slides for, related research papers and Gemini Ai tool.

Google BigQuery/Dremel

- A system that has been in production for **more than 10 years**.
- Goal: present the concept of the system and *map concrete ideas to practice in industry*.

Challenges:

- **Disaggregated resources:** Stateless compute accesses remote shared storage, so I/O and network costs dominate instead of local buffer-cache effects [2, 3].
- **Lack of statistics:** Many queries run *in situ* on heterogeneous external files (CSV/JSON/Parquet) with little or no precomputed statistics, limiting classic cost-based optimization and forcing heuristic/adaptive techniques [1, 3].
- **Columnar and nested data:** Columnar encodings for semi-structured and nested (non-relational) data enable fast scans and compression but complicate query processing and meta-data management [2, 10].

1 Google Dremel and In-Situ Data Processing

Dremel - developed around 2006 as an internal side-project at Google for interactive, ad-hoc SQL queries over log and artifact files produced by other systems; the first version focused on read-only analysis and did *not* support joins [2, 3]. In the late 2010s the engine was rewritten as a shared-disk architecture on top of GFS/Colossus, and Google exposed it externally as the commercial BigQuery service starting in 2012 [3].

By *in-situ* data processing, Dremel/BigQuery executes queries directly on files stored in the data lake without first loading them into a separate DBMS-managed store, then BigQuery acts as a *lakehouse* that adds schemas and SQL for petabyte-scale, interactive latency (seconds rather than minutes/hours) and minimizing data preparation work [2].

Data Systems at Google

- **MapReduce (2004):** processing framework and programming model for large-scale key/value data, based on user-defined *map* and *reduce* functions over distributed files [4].
- **BigTable (2005):** distributed, column key-value store on top of GFS, using LSM-style storage and ordered row keys for scalable range access [5].
- **Chubby (2006):** highly available lock and naming service that stores small metadata items and provides coordination primitives for other Google systems (similar to ZooKeeper) [6].

- **LevelDB (2011):** embedded key–value store library that uses log-structured merge trees for efficiency.
- **Megastore (2010):** semi-relational storage system that layers transactional semantics and replicated entity groups on top of BigTable [7].
- **Vitess (2010):** middleware layer that shards and scales large web workloads.
- **Dremel (2011):** interactive, distributed SQL engine over columnar, nested data in shared storage, later exposed externally as BigQuery [2, 3].
- **Spanner (2011):** globally distributed relational database that provides transactions and synchronous replication [8].
- **F1(2013):** distributed SQL system for Google’s advertising data, built on Spanner [9].
- **Mesa (2014):** large-scale OLAP system for incrementally maintained aggregates over massive datasets, used for metrics and reporting.
- **Napa (2021):** newer internal analytical engine

1.1 Separation of Storage and Compute

Resource disaggregation means independent pools of:

- Storage (Colossus, object store).
- Compute (stateless workers).
- Shuffle memory (separate nodes).

BigQuery user tables are **not stored on the same physical machines** that execute queries, data resides in *Colossus*, Google’s distributed file system. Colossus exposes a **shared, replicated namespace** for files and stores BigQuery tables in a **columnar format** (ColumnIO, later Capacitor) It is responsible for **chunking, replication, and placement** of data across many storage servers. The decoupling of storage and compute allows them to **scale independently** [2, 3, 11, 12].

1.2 Capacitor Columnar Format

Dremel uses **Capacitor**, a columnar encoding for **nested and semi-structured data** [3]. **Repetition / definition levels** in each column so reading a column does not require fetching ancestors in the hierarchy [2]. Access libraries with built-in filtering, similar to Parquet/ORC.

1.3 Zone Maps + Filters + Inverted Indexes

Zone maps: per-file/per-group min/max metadata on a key or set of attributes. Instead of B-trees, Dremel uses these to quickly skip stripes/files that cannot match predicates. Zone maps are hierarchical: table → files → row groups, each with ranges for attributes, ranges must be ordered, similar to ordered key-value layouts in BigTable and other systems [5].

Filters work even with compressed data via dictionary encoding and run-length encoding (RLE). Allow evaluation of predicates without full decompression where possible [10].

Dremel uses **search-style inverted indexes** only, mainly for **text search** over columns where there is no pre-sorted key.

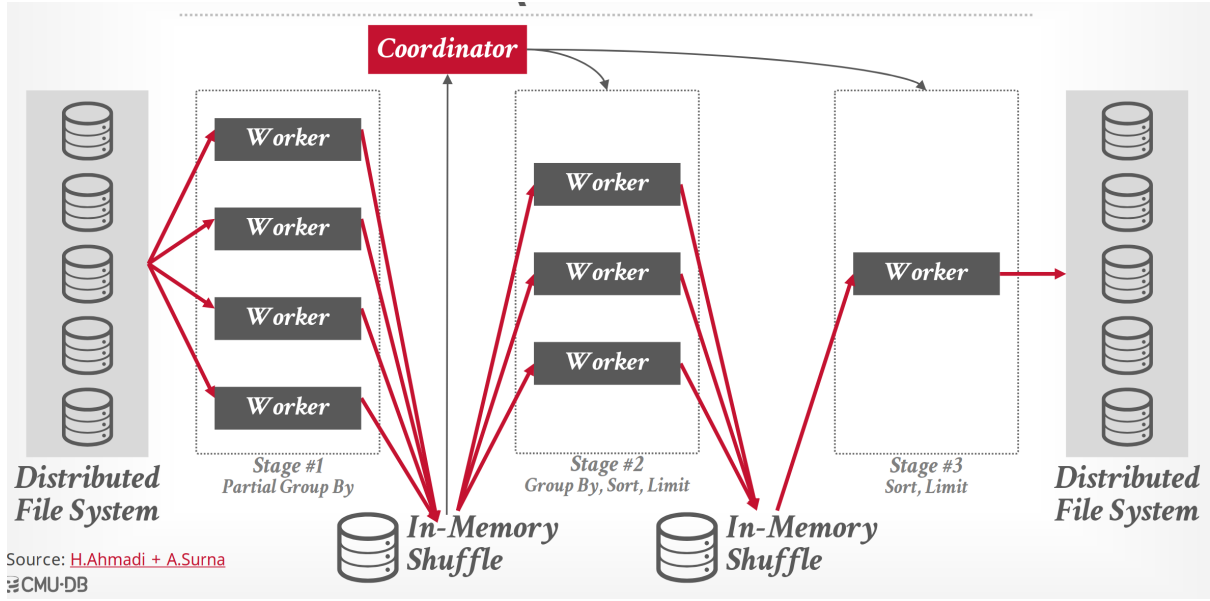


Figure 1: Disaggregated In-Memory Shuffle

2 Execution Model

2.1 Disaggregated In-Memory Shuffle

Starting around 2014, Dremel introduced a dedicated, external shuffle service in memory as shown on picture above [3].

- The logical plan is compiled into **stages** (pipelines), leaf stages perform local work (scans, partial aggregations), subsequent stages consume repartitioned intermediate results [2].
- Each stage consists of many **parallel tasks** (deterministic and idempotent) executing the same pipeline on different partitions of data.
- Coordinator retrieves metadata (from Colossus, schema info and uses services like Chubby/ZooKeeper for coordination [6]). Embeds file lists and metadata into the plan and dispatches work to (stateless) workers which have local memory and may spill to local disk if necessary.
- Base table data in Colossus is stored as **large chunks** (tens of MB), each replicated on **at least three servers** with a master that tracks file metadata and the mapping from files to chunks [1].
- Shuffle nodes maintain **statistics** about the sizes and distributions of intermediate results, which are later used by the optimizer [3, 1].
- Support **dynamic resource allocation**:
 - The system can increase/decrease the number of workers for the next stage depending on the size of the shuffle output. Store per-partition buffers in memory and spill to disk only if necessary.
 - Can **repartition** intermediate data when skew is detected (dynamic repartitioning).

2.2 Example

For a query like:

```
SELECT language, MAX(views)
FROM wikipedia
WHERE title LIKE '%Pavlo%'
GROUP BY 1
ORDER BY 2 DESC
LIMIT 100;
```

a possible physical plan is:

1. **Stage 1 – Scan + Partial Group-By** Workers scan Capacitor/Parquet files and apply filters/predicates while reading. Perform **partial aggregation** (local MAX per language).
2. **Shuffle 1 – In-Memory Shuffle** Intermediate results are partitioned by group key and sent to shuffle nodes.
3. **Stage 2 – Global Group-By + Sort + Limit (distributed)** Workers consume partitions from shuffle nodes, finish the aggregation, and perform distributed sort/limit.
4. **Shuffle 2 → Stage 3 – Final Sort/Limit (small)** Final top- k processing, often with far fewer workers.

3 Dremel Query Optimization

Dremel’s optimizer first builds a physical plan using a mix of **rules** and **cost-based** decisions [3, 1]. Rules handle things like predicate pushdown and basic join ordering, while cost-based choices are made only for tables where good statistics exist (e.g., materialized views). For many in-situ files, statistics are missing or unreliable.

4 Adaptive Query Optimization and Joins

After each stage finishes, Dremel looks at the real sizes of the intermediate results and can adjust the plan for the next stage [3, 1]. For joins, it may keep a **shuffle hash join** (both sides partitioned on the join key) or switch to a **broadcast join** when one table turns out to be small, sending that small table to all workers instead of shuffling both sides.

5 Dynamic Repartitioning

Dremel also watches for **skew** when some shuffle partitions grow much larger than others [3, 1]. If a partition is too big, the coordinator tells workers to use a new hash function or additional partitions so that the heavy keys are split up, which balances the load and avoids slow partitions.

6 SQL Layer and Influence on Other Systems

GoogleSQL and ZetaSQL

In the early 2010s, the **GoogleSQL** project unified the many SQL dialects used by different Google DBMSs into a single language (data model, type system, syntax, semantics, functions), and its open-source **ZetaSQL** exposes much of the same semantics for external use.

Systems Inspired by Dremel

Several distributed SQL engines and shuffle services are explicitly inspired by Dremel, including **Apache Drill**, **PrestoDB/Trino**, **Apache Impala**, and **Dremio**, as well as remote shuffle-as-a-service systems such as **Apache Celeborn** and related work, all of which adopt Dremel-like ideas for SQL-on-lake execution, columnar/vectorized processing, and externalized shuffle.

References

- [1] A. Pavlo. Google BigQuery / Dremel. Lecture slides for CMU 15–721: Advanced Database Systems, Spring 2024.
- [2] S. Melnik et al. Dremel: Interactive Analysis of Web-Scale Datasets. In *Proc. VLDB*, 2010.
- [3] A. Tirosch et al. Dremel: A Decade of Interactive SQL Analysis at Google. In *Proc. VLDB*, 2020.
- [4] J. Dean and S. Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. In *Proc. OSDI*, 2004.
- [5] F. Chang et al. Bigtable: A Distributed Storage System for Structured Data. In *Proc. OSDI*, 2006.
- [6] M. Burrows. The Chubby Lock Service for Loosely-Coupled Distributed Systems. In *Proc. OSDI*, 2006.
- [7] J. Baker et al. Megastore: Providing Scalable, Highly Available Storage for Interactive Services. In *Proc. CIDR*, 2011.
- [8] J. C. Corbett et al. Spanner: Google’s Globally-Distributed Database. In *Proc. OSDI*, 2012.
- [9] J. Shute et al. F1: A Distributed SQL Database that Scales. In *Proc. VLDB*, 2013.
- [10] D. J. Abadi, P. A. Boncz, and S. Harizopoulos. Column-Oriented Database Systems. *VLDB*, 2(2): 1664–1675, 2009.
- [11] Jordan Tigani and Slava Chernyak. *Google BigQuery Analytics*. Wiley, 2014.
- [12] Google Cloud. BigQuery: A fully managed, petabyte-scale data warehouse. Technical whitepaper, Google Cloud, 2016 (and subsequent updates).